CivicMind AI — Smart Governance Platform

Technical & Functional Report

Project: AI-Enabled Civic Issue Detection, Routing, and Resolution
Document type: System design, workflows, and implementation reference
Version: 1.0 (aligned with `feature/civicmind-auth-multi-issue` codebase)

---

1. Executive Summary

CivicMind AI is a full-stack web application that lets citizens report public infrastructure problems (potholes, garbage pileups, and manhole defects) using photos, text descriptions, and GPS coordinates. The platform uses multimodal artificial intelligence—computer vision and natural language processing—to classify issues, estimate confidence scores, and automatically route tickets to the correct municipal departments. When confidence is insufficient, especially in multi-issue photos, reports enter a **manual review queue** where administrators inspect evidence and assign departments.

The system supports **citizen** and **admin** roles, persistent ticket storage (MongoDB Atlas or local JSON fallback), department email notifications, analytics dashboards, and optional on-device or cloud vision models. Design goals include sub-minute citizen submission, transparent scoring, and audit-friendly status history on every ticket.

---

2. Problem Statement & Objectives

Municipal complaint systems often rely on manual triage: citizens call hotlines or fill generic forms, and staff must guess issue type and owning department. CivicMind addresses this by:

1. **Automating visual diagnosis** from citizen-uploaded photos.
2. **Detecting multiple coexisting issues** in one image (e.g., pothole + garbage + damaged manhole).
3. **Routing or escalating** based on configurable confidence thresholds.
4. **Providing role-based portals** for citizens, administrators, and department staff.
5. **Maintaining a durable ticket lifecycle** from submission through resolution.

Success is measured by routing accuracy, time-to-submit, manual review rate, and department response tracking—not by replacing human judgment in ambiguous cases.

---

3. Technology Stack

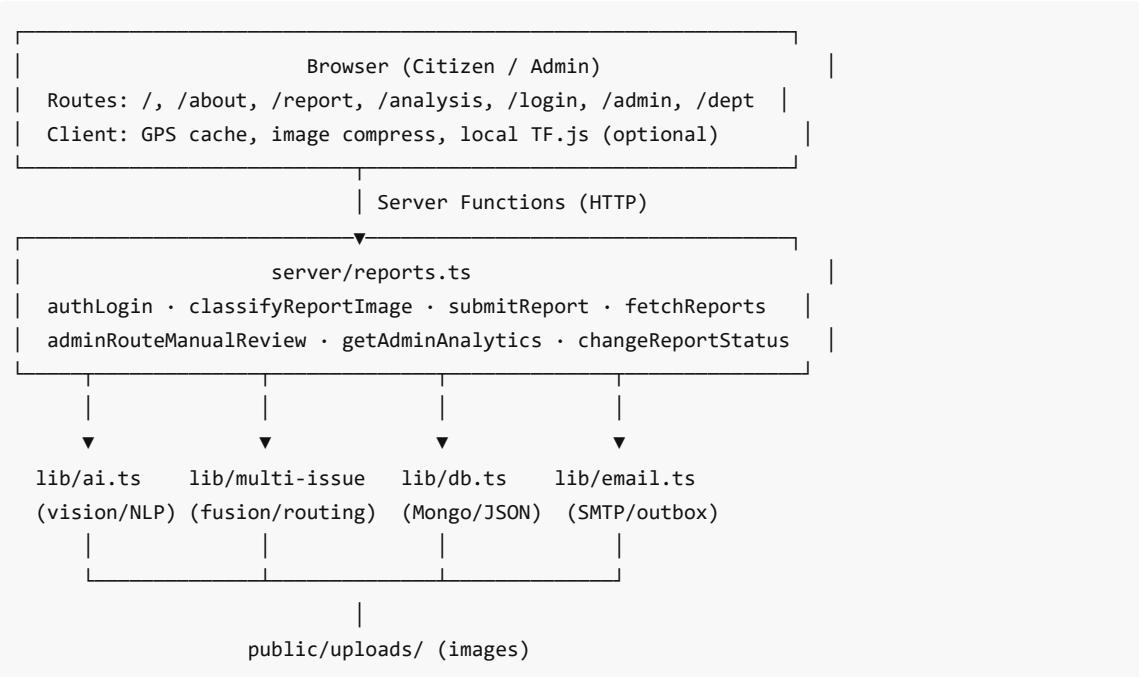
Layer	Technology
Framework	TanStack Start (React 19, file-based routing)
Build	Vite
Styling	Tailwind CSS 4, shadcn/ui components
Validation	Zod (forms and server function inputs)
Server API	TanStack `createServerFn` (RPC-style handlers in `server/reports.ts`)
Database	MongoDB Atlas when `MONGODB_URL` is set; else `data/store.json`

AI (cloud)	Cohere Vision/NLP, optional OpenAI Vision
AI (local)	TensorFlow.js MobileNet model in `public/models/civic-classifier/`
Email	SMTP (e.g., Gmail) with local outbox fallback
Auth	HMAC-signed session tokens, `sessionStorage` on client

Configuration secrets live in **`env/civicmind.config`** (gitignored). A template is provided as `env/civicmind.config.example`.

---

4. High-Level Architecture



Request flow for a typical photo report:

1. Citizen uploads image on `/report` → client compresses (max 1024px, JPEG ~0.8).
2. `classifyReportImage` runs vision AI → UI shows per-class scores.
3. On submit, `submitReport` reuses cached classification when fingerprint matches → saves image → fuses scores → decides route vs manual review → inserts ticket → queues emails asynchronously.

---

5. User Roles & Application Routes

5.1 Citizen

After login (`/login`), citizens access:

Route	Purpose
`/`	Home, platform stats, marketing content
`/about`	Product explanation

`/report`	Submit issue (photo and/or description + GPS)
`/analysis`	Latest submission scoring breakdown
`/reports/:id`	Ticket detail and status timeline

Protected by `AuthGate` in the civic layout; unauthenticated users redirect to `/login`.

5.2 Administrator

Admins have citizen routes plus:

Route	Purpose
`/admin`	Analytics dashboard, manual review queue, all tickets, email outbox

Admin APIs require `authToken` with role `admin` (see Section 10).

5.3 Department staff

Route	Purpose
`/dept`	Department-scoped queue (Public Works, Sanitation, Sewer Maintenance)

---

6. Issue Types & Departments

Three issue types are modeled in `lib/constants.ts` (`ISSUE_META`):

Issue type	Label	Department	Example keywords
`pothole`	Pothole	Public Works	road, crack, asphalt, crater
`garbage`	Garbage Pileup	Sanitation	trash, waste, litter, dump
`manhole`	Manhole Defect	Sewer Maintenance	sewer, drain, cover, missing

Each type has UI colors, base vision/NLP prior scores, and keyword lists for heuristic fallback when cloud vision is unavailable.

---

7. Citizen Report Flow (Detailed)

7.1 Input requirements

- **Photo only:** Valid if image is provided (description optional).
- **Text only:** Description must be at least 10 characters.
- **Both:** Vision and NLP modalities are fused (70% vision, 30% NLP by default).

7.2 Location (GPS)

On page load, the app requests location once with **low accuracy**, **5 second timeout**, and **5 minute cache** in `sessionStorage`. Coordinates appear as the location label (e.g., `GPS · 28.61390°, 77.20900°`). Submission is **not blocked** while GPS is still loading if cached coordinates exist; user can tap **Refresh location**. Reverse geocoding is not required for submit.

7.3 Vision preview (before submit)

When a photo is selected:

1. Image is compressed client-side to reduce upload latency.
2. Server `classifyReportImage` returns `ImageClassificationResult`: primary type, confidence, reason, `perClassScores` for all three classes, model source (`cohere`, `openai`, `local`, `heuristic`).
3. If **two or more** classes score ≥ **8%** (`ISSUE_CANDIDATE_MIN_CONFIDENCE`), UI shows **Multiple issues detected** with bars per type.
4. If any visible issue is below **24%** (`MULTI_ISSUE_MIN_CONFIDENCE`), UI warns that submit will go to **manual review**.

Optional browser **local MobileNet** model may override weak server heuristics when trained (`ml/` pipeline).

7.4 Submit payload

`submitReport` receives `FormData`: issue type (optional on multi-issue), description, lat/lng, label, image, optional `visionClassification` JSON + `imageFingerprint` to skip duplicate vision API calls.

7.5 Post-submit

User is navigated to `/analysis` with ticket ID. Toast reflects auto-route vs manual review vs multi-department alert.

---

8. Artificial Intelligence Pipeline

8.1 Vision classification chain

Implemented in `lib/ai.ts` → `classifyIssueFromImage`:

1. **Cohere Vision** (`COHERE_API_KEY`, model e.g. `command-a-vision-07-2025`) — JSON with `perClassScores`.
2. **OpenAI Vision** if Cohere fails.
3. **Local TF.js model** (`lib/local-vision.ts`, assets under `public/models/`).
4. **Heuristic** keyword match on filename/description (lowest accuracy).

Cloud calls use **15 second timeouts** (`lib/fetch-with-timeout.ts`) with fallback to next tier.

The vision prompt explicitly asks the model to score **all visible issue types**, enabling multi-issue detection in a single photo.

8.2 Description classification

Text-only reports use keyword scoring (`classifyIssueFromDescription`) or cloud NLP when integrated into the submit path via description-derived scores in `lib/multi-issue.ts` (`scoresFromDescription`).

8.3 Score fusion

`fuseModalityScores` in `lib/multi-issue.ts`:

- If both vision and NLP present: `fused[type] = 0.7 × vision + 0.3 × NLP` (normalized).
- Photo-only: vision scores only.
- Text-only: NLP scores only.

8.4 Analysis report (`analyzeReport`)

Produces display scores stored on the ticket:

- **Vision** from classification confidence.
- **NLP:** For photo-only submissions, NLP is stored as **0** and shown as **N/A** in UI; final score equals vision.
- **With description:** Cohere NLP or keyword heuristic; final = weighted sum.

Important: Ticket **routing status** (`routed` vs `manual_review`) is determined by `summarizeMultiIssue` on the fused breakdown, not solely by `analyzeReport.routed` .

---

9. Routing Rules & Thresholds

Central constants in `lib/constants.ts` :

Constant	Value	Meaning
`ROUTING_THRESHOLD`	0.45 (45%)	Single-issue auto-route if primary fused confidence ≥ this
`MULTI_ISSUE_MIN_CONFIDENCE`	0.24 (24%)	Each detected issue must meet this for **automatic** multi-department routing
`ISSUE_CANDIDATE_MIN_CONFIDENCE`	0.08 (8%)	Minimum to show/count as a visible issue in UI and breakdown
`VISION_WEIGHT` / `NLP_WEIGHT`	0.7 / 0.3	Multimodal fusion weights

9.1 Single-issue path

One dominant issue above 8% and not treated as multi-issue → auto-route if primary ≥ 45%; else **manual_review**.

9.2 Multi-issue path

Two or more candidates ≥ 8%:

- **Auto-route** to all distinct departments only if **every** candidate ≥ 24%.
- Otherwise → **manual_review** with full breakdown stored for admin (e.g., garbage 69%, pothole 23%, manhole 8% → admin routes after review).
- **No department emails** sent until auto-routed or admin confirms route from manual review.

9.3 Manual override

Citizen can pick issue type manually; submit boosts chosen type in breakdown (`manualOverride` flag).

---

10. Authentication & Authorization

- Login: username/password checked against env-configured credentials (`AUTH_CITIZEN_*` , `AUTH_ADMIN_*`).
- Session: HMAC-SHA256 token (`AUTH_SESSION_SECRET`), 7-day expiry, stored in browser `sessionStorage` .
- Server functions call `assertRole(token, "admin")` for admin-only lists, analytics, manual review routing.
- Citizens cannot access `/admin` navigation unless logged in as admin.

Default dev credentials exist only when env vars are unset; production must set strong secrets in `civicmind.config` .

---

11. Data Model & Persistence

11.1 Report record

Key fields (`lib/types.ts` → `ReportRecord`):

- `id` (e.g., `CIV-192496`), `issueType` , `description` , `imagePath`
- `lat` , `lng` , `locationLabel`
- `visionScore` , `nlpScore` , `finalScore` , `routed` , `status`
- `isMultiIssue` , `issueBreakdown[]` , `notifiedDepartments[]`
- `lastStatusNote` (e.g., manual review reason)
- Timestamps `createdAt` , `updatedAt`

11.2 Status lifecycle

`submitted` → `routed` or `manual_review` → `in_progress` → `resolved` → `closed`

History entries in `statusHistory` collection / store array.

11.3 MongoDB vs local file

- **`MONGODB_URL` set and reachable:** All CRUD via MongoDB; `_id` stripped before API responses (serialization safety).
- **Fallback:** `data/store.json` with reports and history arrays.
- Demo seed runs once on empty database (`seedDemoReports`).

---

12. Admin Manual Review Workflow

1. Low-confidence or partial multi-issue tickets get `status: manual_review` .
2. `/admin` **Manual Review Queue** lists tickets with thumbnail, AI breakdown badges, and status note.
3. Admin selects **issue type**, **department**, optional note → **Confirm & route**.
4. `routeManualReviewReport` updates ticket to `routed` , sends department email, collapses breakdown to routed issue.

Mongo `_id` stripping fix ensures the queue loads correctly in the browser.

---

13. Email Notifications

`lib/email.ts` :

- Builds HTML per department with breakdown percentages.
- Sends via SMTP when configured; otherwise records in **email outbox** (visible on admin dashboard).
- Multi-department: parallel SMTP attempts per recipient group.
- On auto-route submit, notification runs **asynchronously** so the citizen response is not blocked by mail latency.

Department recipient addresses come from `DEPT_EMAIL_*` env vars.

---

14. Performance Optimizations

Optimization	Benefit

Memoized `bootstrapDb()`	Avoid repeated seed checks per request
Vision classification cache on submit	Eliminates second cloud vision call (~seconds saved)
Client image compression	Faster upload and base64 encoding
Async email after insert	Shorter perceived submit time
15s AI timeouts	Predictable failure → fallback tier
Instrumentation `[submitReport]` lap logs	Measure bootstrap, classify, save, analysis, DB, email

---

15. Machine Learning (Optional Local Training)

Directory `ml/`:

1. Prepare dataset under `ml/dataset/train|val/{pothole,garbage,manhole}/`.
2. Train with Python (`train.py`), export TF.js (`export_tfjs.py`).
3. Deploy model to `public/models/civic-classifier/`.
4. Browser hook `useLocalClassifier` and server `local-vision` can classify without API keys.

Documented in `ml/README.md`.

---

16. Security & Privacy Notes

- Never commit `env/civicmind.config`, `.env`, or `data/store.json` with real citizen data.
- Rotate API keys and SMTP passwords if exposed.
- Session tokens must use a long random `AUTH_SESSION_SECRET` in production.
- Uploaded images stored under `public/uploads/` (gitignored); protect server filesystem in deployment.

---

17. Key Source Files (Reference)

Path	Responsibility
`server/reports.ts`	All server functions, submit pipeline
`lib/ai.ts`	Vision/NLP models and heuristics
`lib/multi-issue.ts`	Fusion, breakdown, routing summary
`lib/db.ts`	Persistence, manual review routing
`components/civic/civic-sections.tsx`	Report UI, GPS, classification display
`routes/admin/index.tsx`	Admin dashboard and review cards
`lib/auth.ts`	Login and token validation
`lib/email.ts`	Department notifications

---

18. Conclusion

CivicMind AI combines modern web engineering with practical AI governance workflows: citizens get fast, guided reporting; the system transparently scores evidence; automation handles clear cases; humans retain control for ambiguous multi-issue scenes. Threshold-based routing, manual review, and full audit history make the platform suitable for demos, coursework, and as a foundation for production municipal pilots—with MongoDB, SMTP, and cloud vision configured per environment.

Appendix A — Submit Pipeline (Step-by-Step)

1. **Validate** FormData (photo or ≥10 char description, lat/lng, label).
2. **Bootstrap DB** (once per server process): placeholder image, seed if empty.
3. **Classify:** Reuse client `visionClassification` if fingerprint matches; else call `classifyIssueFromImage`.
4. **Fuse** vision (+ NLP if description present) → `buildIssueBreakdown`.
5. **Summarize** multi-issue vs single; set `routed` or `manual_review`.
6. **Save** image to `public/uploads/`.
7. **Analyze** for display scores (`analyzeReport`).
8. **Generate** ticket ID (`CIV-######`).
9. **Insert** report + status history note.
10. **Queue** department emails (if auto-routed) without blocking response.
11. **Return** `AnalysisResult` to client.

Server logs `[submitReport] <stage>: <ms>` for performance diagnosis.

Appendix B — Ticket Status Reference

Status	Meaning
`submitted`	Legacy/initial (most flows go directly to routed or manual_review)
`routed`	Assigned to department(s); notifications sent or queued
`manual_review`	Awaiting admin triage
`in_progress`	Department actively working
`resolved`	Work completed
`closed`	Ticket archived

Appendix C — Environment Variables (Template)

See `env/civicmind.config.example` for full list:

- **Auth:** `AUTH_SESSION_SECRET`, `AUTH_CITIZEN_*`, `AUTH_ADMIN_*`
- **Database:** `MONGODB_URL`, `MONGODB_DB_NAME`
- **AI:** `COHERE_API_KEY`, `COHERE_VISION_MODEL`, `COHERE_NLP_MODEL`, `OPENAI_API_KEY`
- **Email:** `SMTP_*`, `DEPT_EMAIL_*`, `NOTIFY_EMAIL_*`

Copy example to `env/civicmind.config` and fill values locally (never push secrets to Git).

Appendix D — Example Multi-Issue Scenario

Photo contains: garbage pile (strong signal), pothole (moderate), manhole (weak).

Vision `perClassScores` (illustrative): garbage 69%, pothole 23%, manhole 8%.

UI: Shows all three with progress bars; warns manual review because pothole & manhole < 24%.

Submit result: `status = manual_review`, `isMultiIssue = true`, breakdown stored, no auto emails.

Admin: Opens queue, views photo + badges, routes to Sanitation (or multiple actions via primary choice), ticket becomes `routed`, email sent.

Appendix E — Deployment Checklist

1. Set production secrets in `env/civicmind.config`.
2. Configure MongoDB Atlas network access.
3. Verify SMTP for department addresses.
4. Run `npm run build` && `npm run start` (or host on Node-compatible platform).
5. Ensure `public/uploads` is writable and backed up.
6. Test citizen submit, admin manual review, and dept queue with test accounts.

End of report — CivicMind AI